



UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

► **Curso: Introducción a la Computación**

► **Docente: Américo Chulluncuy Reynoso**

2026-1

Sesión 10:

Arreglos Bidimensionales (matrices)

Contenido

1. Arreglos Bidimensionales
declaración y representación en memoria
2. Inicialización
Estática (en la declaración) vs dinámica (en tiempo de ejecución)
3. Recorrido de Matrices
Bucles anidados, acumuladores por fila, columna y total
4. Arreglos Multidimensionales
5. Búsqueda Lineal
6. Búsqueda binaria

¿Por qué matrices?

Las matrices son la estructura fundamental en gráficos, IA y simulaciones físicas.

Imágenes Digitales

Cada píxel es una celda. Una imagen 1920×1080 es una matriz de 2 millones de valores de color.

Álgebra Lineal

Sistemas de ecuaciones, transformaciones geométricas y rotaciones 3D usan multiplicación matricial.

Machine Learning

Pesos de redes neuronales, datasets y gradientes se representan como matrices (tensores).

Simulaciones

Difusión de calor, propagación de ondas y modelos climáticos discretizan el espacio en grids 2D/3D.

1. Declaración de arreglos bidimensionales

- En la declaración se utilizan 2 índices

```
int exams[4][3];
```

**Número
de filas**

The diagram consists of two blue-outlined ovals at the bottom. The left oval contains the text 'Número de filas'. A blue arrow points from the top of this oval to the green number '4' in the code 'int exams[4][3];'. The right oval contains the text 'Número de columnas'. A blue arrow points from the top of this oval to the green number '3' in the same code.

**Número de
columnas**

Representación visual: `int exams[4][3];`

columns			
f i l a s	<code>exams[0][0]</code>	<code>exams[0][1]</code>	<code>exams[0][2]</code>
	<code>exams[1][0]</code>	<code>exams[1][1]</code>	<code>exams[1][2]</code>
	<code>exams[2][0]</code>	<code>exams[2][1]</code>	<code>exams[2][2]</code>
	<code>exams[3][0]</code>	<code>exams[3][1]</code>	<code>exams[3][2]</code>

Acceso a sus elementos: `exams[2][2] = 19;`

Importante: En memoria se almacena una fila a continuación de otra

2. Inicialización de Matrices

1. Estática completa

```
int notas[2][3]={  
    {11,14,15},  
    {19,13,16}  
};
```

2. Sin llaves internas

```
int notas[2][3]={  
    11,14,15,  
    19,13,16  
};
```

3. Relleno con ceros

```
int notas[2][3]={0};
```

4. Inicialización dinámica (entrada del usuario)

```
for (int i = 0; i < 2; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        cin >> notas[i][j];  
    }  
}
```

3. Recorrido de una matriz

```
const int FILAS = 3, COLS = 3;
int M[FILAS][COLS] = {{1,2,3},{4,5,6},{7,8,9}};

int sumTotal = 0;

for (int i = 0; i < FILAS; i++) {           // itera filas
    int sumFila = 0;
    for (int j = 0; j < COLS; j++) {        // itera columnas
        cout << M[i][j] << " ";           // imprime alineado
        sumFila += M[i][j];
        sumTotal += M[i][j];
    }
    cout << " | suma fila " << i << " = " << sumFila << "\n";
}

cout << "Suma total = " << sumTotal << "\n";
```

- Diagonal principal

Condición: $i == j$

Recorre los elementos
solo de la diagonal

- Diagonal secundaria

Condición: $i + j = n - 1$

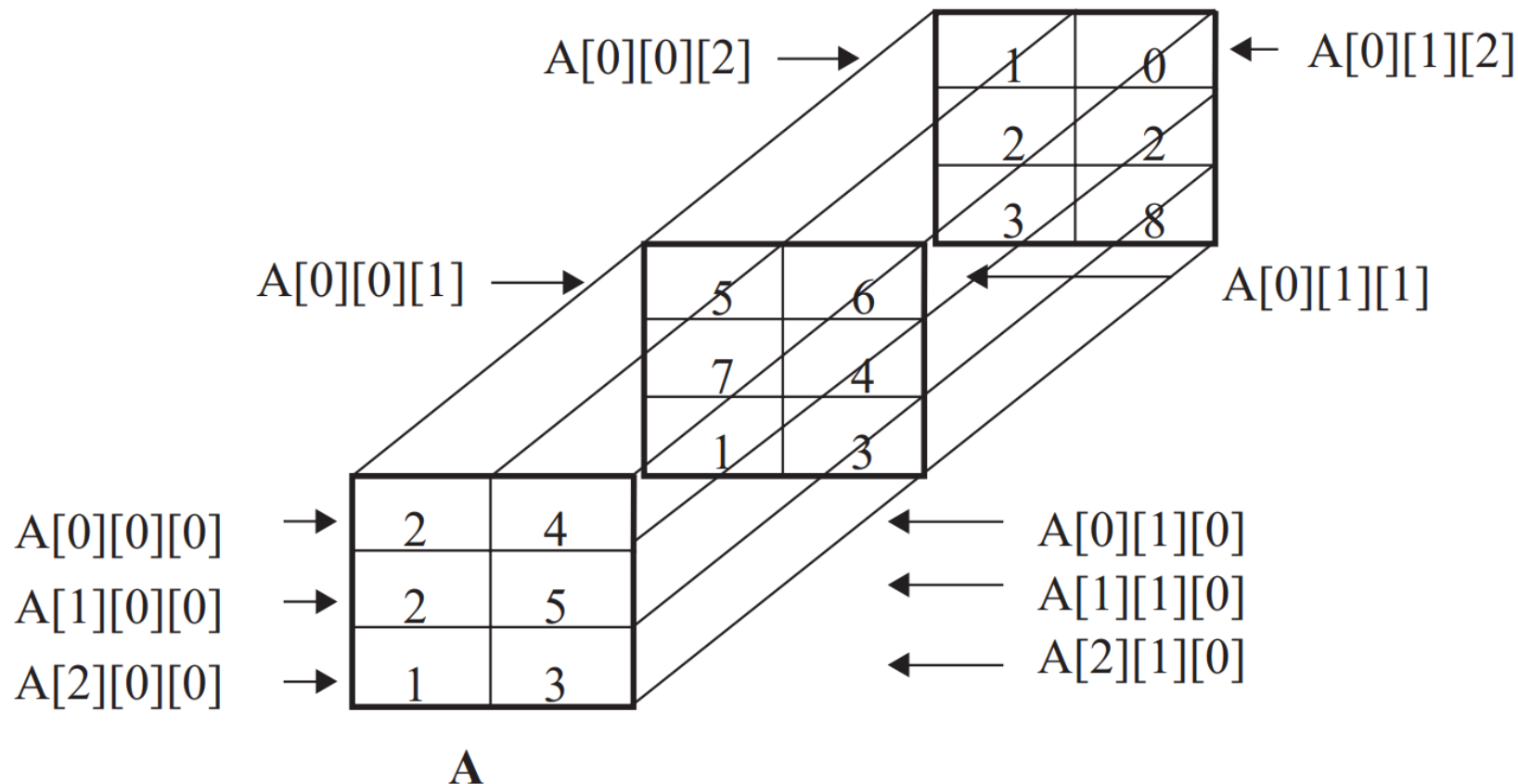
Recorre los elementos
de la diagonal
secundaria

4. Arreglos multidimensionales

- Un arreglo de N dimensiones se declara con N pares de corchetes, cada dimensión añade un nivel de indexación

`int A[3][2][3];`

`double b[3][4][3][4];`



5. Algoritmo de búsqueda lineal

1

Inicializar: Establecer índice = -1 (indica "no encontrado")

2

Recorrer: Comparar cada elemento $a[i]$ con la clave buscada

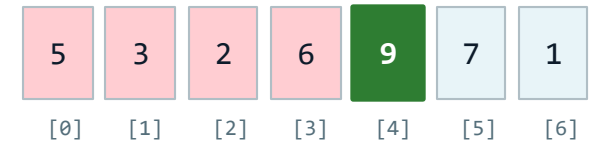
3

Comparar: Si $a[i] == \text{clave}$ → guardar índice i y detener con break

4

Resultado: Si índice $\neq -1$ → éxito; si índice $== -1$ → no existe

Traza visual (clave = 9)



● comparado ● encontrado en [4]

Peor caso: n comparaciones (elemento al final o no existe)

Mejor caso: 1 comparación (primer elemento)

Promedio: $n/2$ comparaciones

6. Algoritmo de búsqueda binaria

1. Divide un arreglo ordenado en tres secciones
 - Elemento medio
 - Elementos al lado izquierdo del elemento medio
 - Elementos al lado derecho del elemento medio
2. Si el elemento medio es el valor correcto, terminamos. En caso contrario, retornamos al paso 1, utilizando solo la mitad del arreglo que puede contener el valor correcto
3. Continuar con los pasos 1 y 2 hasta que se encuentre el valor o no haya más elementos para examinar

Traza paso a paso (search = 23)

Search 23	0	1	2	3	4	5	6	7	8	9
	2	5	8	12	16	23	38	56	72	91

23 > 16 take 2 nd half	L=0	1	2	3	M=4	5	6	7	8	H=9
	2	5	8	12	16	23	38	56	72	91

23 < 56 take 1 st half	0	1	2	3	4	L=5	6	M=7	8	H=9
	2	5	8	12	16	23	38	56	72	91

Found 23, Return 5	0	1	2	3	4	L=5, M=5	H=6	7	8	9
	2	5	8	12	16	23	38	56	72	91

Comparación: Lineal $O(n)$ vs Binaria $O(\log n)$ Para $n=1,000,000$:

lineal = 1,000,000 comparaciones | binaria ≤ 20 comparaciones

Búsqueda binaria en C++

```
int a[] = {2,5,8,12,16,26,38,56,72,91};
int tam = sizeof(a)/sizeof(a[0]);

int clave = 12;
int indice = -1;

int izq = 0;
int der = tam - 1;

while(izq <= der){
    int medio = (izq + der)/2;
    if(a[medio] == clave){
        indice = medio;
        break;
    }else if(a[medio] < clave){
        izq = medio +1;
    }else{
        der = medio -1;
    }
}

if(indice != -1){
    cout << "El valor " << clave << " se encuentra en el índice " << indice <<endl;
}else{
    cout << "El valor " << clave << " no se encuentra en el arreglo";
}
```

Ejemplo Integrador

Almacenar notas de 3 alumnos en 4 materias. Calcular promedio por alumno, promedio por materia y el mejor estudiante.

Ejercicios

1. Escribir un programa que decida si una matriz es simétrica
2. Escribir un programa calcule la suma de dos matrices e imprima la matriz resultante.
3. Escribir un programa que encuentre el mayor y menor elemento de una matriz.
4. Escribir un programa que muestre la transpuesta de una matriz dada.
5. Escribir un programa que calcule el producto de 2 matrices e imprima la matriz resultante